

Chahinez Benaicha

LAB 5 RAPPORT

Directory Facilitator (DF) agent and Services

The Directory Facilitator (DF)

The DF is a system agent integrated into JADE that implements the Yellow Pages service. Its role is to register and search for agents based on the services they offer. An agent can:

- Register with a service type and name via **DFService.register()**
- Search for agents offering a given service via **DFService.search()**
- Deregister at the end of its lifecycle via **DFService.deregister()**

Exercise 1 – Communication BuyerAgent / SellerAgent

Establish communication between a buyer agent and a seller agent via the DF. The buyer sends a request to obtain the list of available cars; the seller responds with this list in the form of a serialized object:

- **BuyerAgent**: registers with the DF, sends a request to seller1 using its AID, receives the list, and sends "Bye".
- **SellerAgent**: registers with the DF (car-selling service), waits for messages in a cyclic manner, sends the serialized list of cars, and terminates upon receipt of "Bye".

```
// Creation of the Buyer Agent description
DFAgentDescription Afd = new DFAgentDescription();
Afd.setName(getAID());
ServiceDescription sd = new ServiceDescription();
sd.setType("car-buying");
sd.setName("buyer-service");
Afd.addServices(sd);
// Register the agent's description in DF
DFService.register(this, Afd);
System.out.println(getLocalName() + " Registration in the DF directory");
catch (FIPAException e) {
    e.printStackTrace();
}
```

- The Seller agent receives the request for the products list and responds to the Buyer agent by sending to it the product list:

```
addBehaviour(new CyclicBehaviour() {
    public void action() {
        // Awaiting message (from the Buyer agent)
        ACLMessage msg = receive(MessageTemplate.MatchPerformative(ACLMessage.INFORM));
        if (msg != null) {
            if (msg.getContent().equalsIgnoreCase("Request for car lists")) {
                System.out.println("receipt of the customer's request ");
                ACLMessage reply = msg.createReply();
                Object[] cars = {
                    "PORCHE", "LAMBORGHINI", "MASERATI",
                    "CHEVROLET", "LAGUNA", "ROMEO"
                };
                try {
                    reply.setContentObject(cars);
                } catch (IOException e) {e.printStackTrace();}
                // Send the response to the Buyer Agent
                myAgent.send(reply);
                System.out.println("Sending the list of cars to the customer ");
            }
        }
    }
});
```

Result on the console:

```
buyer1 Registration in the DF directory
buyer1 --> Request for car lists
seller1 Agent registered in DF
receipt of the customer's request
Sending the list of cars to the customer
seller1 replied
seller1 cancellation of registration with DF
the list of products is :
PORCHE
LAMBORGHINI
MASERATI
CHEVROLET
LAGUNA
ROMEO
( agent-identifier :name buyer1@192.168.80.1:1099/JADE :addresses (sequence http
buyer1 cancellation of registration with DF
```

CarBuyerAgent (Buyer1) → “request for car seller ” → CarSellerAgent (Seller1) → Available cars list →
CarBuyerAgent (Buyer1) → “Bye” → CarSellerAgent (Seller1) → “Bye”

Exercise 2

Three agents are created in the Lab5.exercice2 package:

- **CarSellerAgent**: publishes the "car-selling" service in the DF. Responds to requests for lists of cars.
- **BookSellerAgent**: publishes the "book-selling" service on the DF. It will not be contacted by the **CarBuyer**.
- **CarBuyerAgent**: searches the DF for the agent offering "car-selling", dynamically retrieves its AID, then sends it the request.

Code snippet — Searching the DF

The `searchCarSellerInDF()` method sends a targeted query to the DF using the "car-selling" service type:
`CarBuyerAgent` → DF : "search car selling" → AID of `CarSellerAgent` → `CarBuyerAgent`

```
private AID searchCarSellerInDF() {
    // Template de recherche : on ne précise que le type de service.
    DFAgentDescription template = new DFAgentDescription();
    ServiceDescription sd = new ServiceDescription();
    sd.setType("car-selling"); // Critère de filtrage
    template.addServices(sd);

    try {
        // Résultats : tableau de DFAgentDescription
        DFAgentDescription[] results = DFService.search(this, template);
        if (results.length > 0) {
            // On extrait l'AID de la première correspondance trouvée
            return results[0].getName();
        }
    } catch (FIPAException e) {
        e.printStackTrace();
    }
    return null; // Aucun vendeur de voitures trouvé dans le DF
}
```

How to use it:

```
//
AID carSellerAID = searchCarSellerInDF();

if (carSellerAID == null) {
    // Si aucun vendeur de voitures n'est trouvé dans le DF
    System.out.println(getLocalName() + " No car seller found in DF. Terminating.");
    doDelete();
    return;
}
System.out.println(getLocalName() + " Found car seller in DF : " + carSellerAID.getLocalName());
```

`CarBuyerAgent` → "Request for Cars list" → `CarSellerAgent` → "Sending the list of cars to the customer" → `CarBuyerAgent` → "Bye" → `CarSellerAgent` (Seller1) → "Bye"

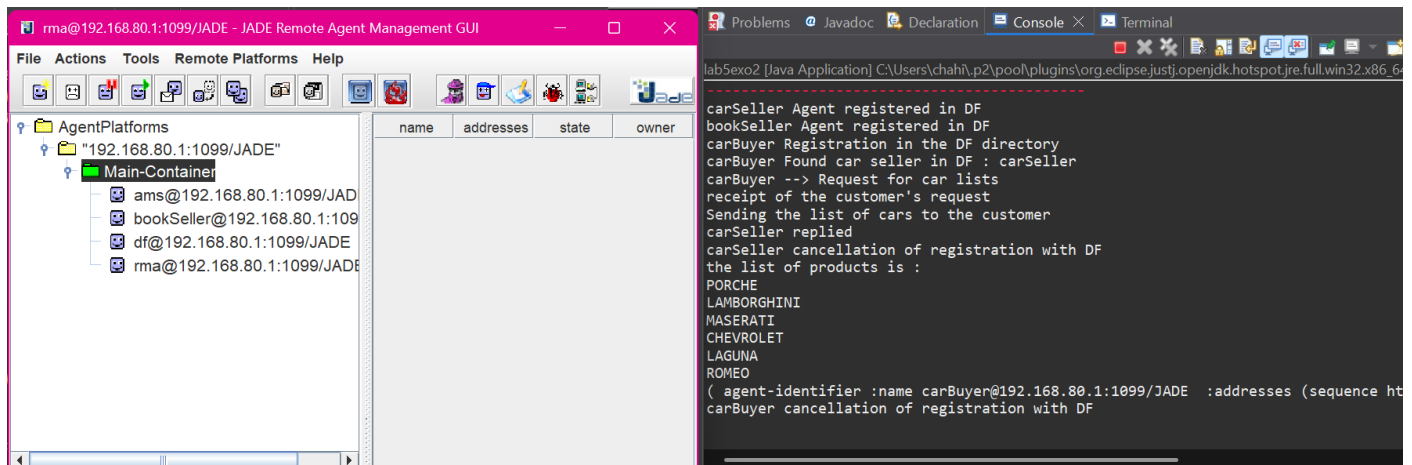
```
ACLMessage msg = new ACLMessage(ACLMessage.INFORM);
msg.setContent("Request for car lists");
msg.addReceiver(carSellerAID); // AID obtenu dynamiquement du DF
send(msg);
System.out.println(getLocalName() + " -> Request for car lists");

addBehaviour(new CyclicBehaviour(this) {
    public void action() {
        ACLMessage msg = receive(MessageTemplate.MatchPerformative(ACLMessage.INFORM));

        if (msg != null) {
            try {
                ACLMessage reply = msg.createReply();
                reply.setContent("Bye");
                send(reply);

                cars = (Object[]) msg.getContentObject();
                System.out.println("the list of products is : ");
                for (int i = 0; i < cars.length; i++) {
                    System.out.println(cars[i]);
                }
                System.out.println(getAID() + " sent Bye");
            } catch (UnreadableException e) {
                e.printStackTrace();
            }
            // Mission accomplie - on se termine.
            doDelete();
        } else {
            block();
        }
    }
});
```

Results on the console:



Conclusion

This practical exercise provided a practical understanding of two fundamental mechanisms of multi-agent systems in JADE:

- Dynamic manipulation & filtering of the DF (the agents start up on their own and each log in to the DF using their own service type).
- ACL communication with serialised objects: using `setContentObject` / `getContentObject` allows complex data structures (arrays, Java objects) to be exchanged between agents.